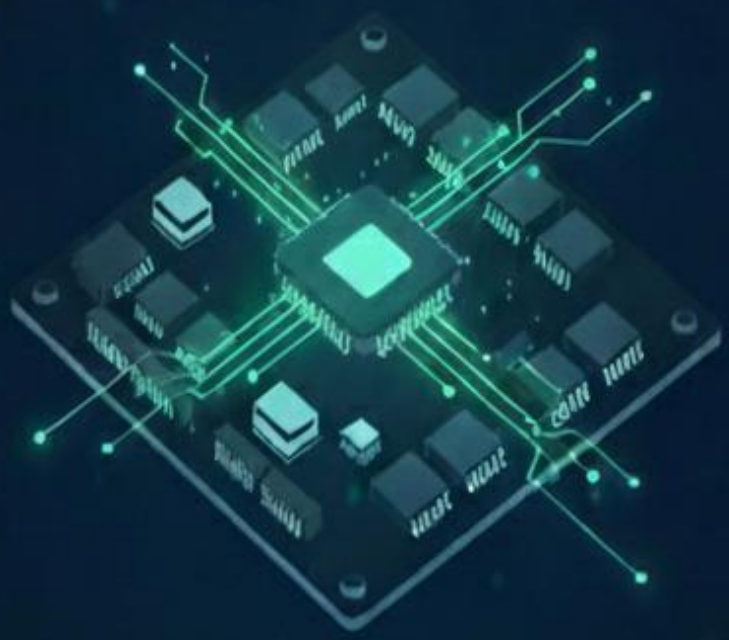
★ MASTERING DIGITAL LOGIC ★

100 Synthesizable Verilog Codes

The Essential RTL Handbook for FPGA & ASIC Design



- ✓ Foundational Blocks
- ✓ Advanced FSM & Piplelining
- ✓ Fully Parameterizable
- ✓ Copy-Paste Ready for Synth

★ GOLD EDITION ★

100 Synthesizable Verilog Code

(Self-contained, synthesisable)

Q1 2-to-1 multiplexer (behavioural)

```
module mux2_1_beh (
    input  a,
    input  b,
    input  sel,
    output y
);
assign y = sel ? b : a;
endmodule
```

Q2 2-to-1 mux (gate-level)

```
module mux2_1_gate (
    input  a, b, sel,
    output y
);
wire sel_n, t1, t2;
not (sel_n, sel);
and (t1, a, sel_n);
and (t2, b, sel);
or  (y, t1, t2);
endmodule
```

Q3 4-to-1 mux (case)

```
module mux4_1_case (
    input      [3:0] i,
    input      [1:0] sel,
    output reg       y
);
always @(*) case(sel)
    2'b00: y = i[0];
    2'b01: y = i[1];
    2'b10: y = i[2];
    2'b11: y = i[3];
endcase
endmodule
```

Q4 Half adder (data-flow)

```
module ha_df (input a, b, output sum, cout);
assign sum  = a ^ b;
assign cout = a & b;
endmodule
```

Q5 Full adder (data-flow)

```
module fa_df (input a, b, cin, output sum, cout);
assign sum  = a ^ b ^ cin;
assign cout = (a & b) | (b & cin) | (cin & a);
endmodule
```

Q6 4-bit ripple adder

```
module add4_ripple (
    input  [3:0] a, b,
    input        cin,
    output [3:0] sum,
    output       cout
);
wire c1, c2, c3;
fa_df f0 (.a(a[0]), .b(b[0]), .cin(cin),  .sum(sum[0]), .cout(c1));
fa_df f1 (.a(a[1]), .b(b[1]), .cin(c1),   .sum(sum[1]), .cout(c2));
fa_df f2 (.a(a[2]), .b(b[2]), .cin(c2),   .sum(sum[2]), .cout(c3));
fa_df f3 (.a(a[3]), .b(b[3]), .cin(c3),   .sum(sum[3]), .cout(cout));
endmodule
```

Q7 4-bit subtractor

```
module sub4 (
    input  [3:0]  a,  b,
    output [3:0]  diff,
    output       borrow
);
wire [3:0] b_inv = ~b;
wire       c_out;
add4_ripple u0 (.a(a), .b(b_inv), .cin(1'b1), .sum(diff), .cout(c_out));
assign borrow = ~c_out;
endmodule
```

Q8 2-bit greater-than comparator

```
module cmp_gt2 (input [1:0] a, b, output gt);
assign gt = (a > b);
endmodule
```

Q9 2-bit equality comparator

```
module cmp_eq2 (input [1:0] a, b, output eq);
assign eq = (a == b);
endmodule
```

Q10 1-bit magnitude comparator

```
module mag1 (input a, b, output gt, eq, lt);
assign gt =  a & ~b;
assign eq = ~(a ^ b);
assign lt = ~a &  b;
endmodule
```

Q11 2-bit binary → gray

```
module bin2gray2 (input [1:0] bin, output [1:0] gray);
assign gray[1] = bin[1];
assign gray[0] = bin[1] ^ bin[0];
endmodule
```

Q12 2-bit gray → binary

```
module gray2bin2 (input [1:0] gray, output [1:0] bin);
assign bin[1] = gray[1];
assign bin[0] = gray[1] ^ gray[0];
endmodule
```

Q13 Half subtractor

```
module hs_gate (input a, b, output dif, bor);
assign dif = a ^ b;
assign bor = ~a & b;
endmodule
```

Q14 Full subtractor

```
module fs_gate (input a, b, bin, output dif, bout);
assign dif  = a ^ b ^ bin;
assign bout = (~a & b) | (~a & bin) | (b & bin);
endmodule
```

Q15 2-bit unsigned multiplier

```
module mul2u (input [1:0] a, b, output [3:0] p);
assign p = a * b;
endmodule
### Q13 Half subtractor
```

Q16 4-bit ALU (add, and, or, xor)

```
module alu4op (
    input  [3:0] a, b,
    input  [1:0] op,
    output reg [3:0] y
);
always @(*) case(op)
    2'b00: y = a + b;
    2'b01: y = a & b;
    2'b10: y = a | b;
    2'b11: y = a ^ b;
endcase
endmodule
```

Q17 Tristate buffer

```
module tribuf1 (input d, en, output y);
assign y = en ? d : 1'bz;
endmodule
```

Q18 4-bit tristate bus

```
module bus4tri (
    input  [3:0] d1, d2,
    input        e1,  e2,
    inout [3:0]  bus
);
assign bus = e1 ? d1 : 8'hzz;
assign bus = e2 ? d2 : 8'hzz;
endmodule
```

Q19 2-to-4 decoder (low outputs)

```
module dec2_4l (input [1:0] a, input en, output [3:0] y);
assign y = en ? ~(4'b0001 << a) : 4'b1111;
endmodule
```

Q20 4-to-2 priority encoder

```
module enc4_2p (input [3:0] r, output reg [1:0] y);
always @(*) begin
    if     (r[3])  y  =  2'b11;
    else if (r[2]) y = 2'b10;
    else if (r[1]) y = 2'b01;
    else           y = 2'b00;
end
endmodule
```

Q21 8-bit even parity

```
module parity_even8 (input [7:0] d, output pe);
assign pe = ^d;
endmodule
```

Q22 4-bit barrel rotate-left

```
module barrel_rotl4 (input [3:0] d, input [1:0] s, output [3:0] y);
assign y = (d << s) | (d >> (4-s));
endmodule
```

Q23 50 MHz → 1 Hz pulse

```
module div50M_1Hz (
    input  clk50, ar,
    output reg tick
);
reg [25:0] cnt;
always @(posedge clk50 or posedge ar) begin
    if (ar) begin cnt<=0; tick<=0; end
    else begin
        if (cnt == 26'd24_999_999) begin cnt<=0; tick<=1; end
        else begin cnt<=cnt+1; tick<=0; end
    end
```

```
end
endmodule
```

Q24 Synchronizer flip-flop

```
module sync_ff (input async_d, clk, output sync_q);
reg q1, q2;
always @(posedge clk) begin q1<=async_d; q2<=q1; end
assign sync_q = q2;
endmodule
```

Q25 Rising-edge detector

```
module rising_det (input d, clk, output pe);
reg q1, q2;
always @(posedge clk) begin q1<=d; q2<=q1; end
assign pe = q1 & ~q2;
endmodule
```

Q26 Falling-edge detector

```
module falling_det (input d, clk, output fe);
reg q1, q2;
always @(posedge clk) begin q1<=d; q2<=q1; end
assign fe = ~q1 & q2;
endmodule
```

Q27 Dual-edge detector

```
module both_edge (input d, clk, output edge);
reg q1, q2;
always @(posedge clk) begin q1<=d; q2<=q1; end
assign edge = q1 ^ q2;
endmodule
```

Q28 Toggle flip-flop

```
module tff (input t, clk, ar, output reg q);
always @(posedge clk or posedge ar)
if(ar) q<=0; else if(t) q<=~q;
endmodule
```

Q29 Divide-by-3 counter (50 % duty)

```
module div3_50 (
    input  clk, ar,
    output reg out
);
reg [1:0] st, nxt;
parameter S0=0, S1=1, S2=2;
always @(posedge clk or posedge ar) if(ar) st<=S0; else st<=nxt;
always @(*) case(st) S0: nxt=S1; S1: nxt=S2; S2: nxt=S0; endcase
always @(*) out = (st==S1);
endmodule
```

Q30 3-bit LFSR (x^3+x^2+1)

```
module lfsr3 (input clk, ar, output [2:0] q);
reg [2:0] r;
always @(posedge clk or posedge ar)
if(ar) r<=3'b001; else r<={r[1],r[0],r[2]^r[1]};
assign q=r;
endmodule
```

Q31 Johnson counter (3-bit)

```
module johnson3 (input clk, ar, output [2:0] q);
reg [2:0] sr;
always @(posedge clk or posedge ar)
if(ar) sr<=3'b000; else sr<={~sr[0],sr[2:1]};
assign q=sr;
endmodule
```

Q32 Ring counter (3-bit)

```
module ring3 (input clk, ar, output [2:0] q);
reg [2:0] sr;
always @(posedge clk or posedge ar)
if(ar) sr<=3'b001; else sr<={sr[0],sr[2:1]};
assign q=sr;
endmodule
```

Q33 Mod-5 counter

```
module cnt_mod5 (input clk, ar, output reg [2:0] q);
always @(posedge clk or posedge ar)
```

```
if(ar) q<=0; else if(q==4) q<=0; else q<=q+1;
endmodule
```

Q34 4-bit up/down counter

```
module cnt4_ud (input clk, up, output reg [3:0] q);
always @(posedge clk)
q <= up ? q+1 : q-1;
endmodule
```

Q35 4-bit loadable counter

```
module cnt4_load (
    input       clk, ar, load, en,
    input [3:0] d,
    output reg [3:0] q
);
always @(posedge clk or posedge ar)
if(ar) q<=0;
else if(load) q<=d;
else if(en)   q<=q+1;
endmodule
```

Q36 Decimal counter (0-9)

```
module cnt_bcd (input clk, ar, output reg [3:0] q);
always @(posedge clk or posedge ar)
if(ar) q<=0; else if(q==9) q<=0; else q<=q+1;
endmodule
```

Q37 Clock mux (glitch-free)

```
module clkmux (
    input  clk1, clk2, sel,
    output clkout
);
reg s1, s2;
always @(posedge clk1) s1<= ~sel;
always @(posedge clk2) s2<= sel;
assign clkout = (clk1 & s1) | (clk2 & s2);
endmodule
```

Q38 Debounce 1 ms @50 MHz

```
module debounce1ms (
    input  btn, clk, ar,
    output reg db
);
reg [19:0] ctr;
reg btn1, btn2;
always @(posedge clk or posedge ar) begin
 if(ar) begin btn1<=0; btn2<=0; ctr<=0; db<=0; end
 else begin
  btn1<=btn; btn2<=btn1;
  if(btn2!=db) ctr<=0;
  else if(ctr==20'd49_999) db<=btn2;
  else ctr<=ctr+1;
 end
end
endmodule
```

Q39 Moore 101 detector

```
module moore101 (
    input x, clk, ar,
    output reg z
);
reg [1:0] st, nxt;
parameter S0=0, S1=1, S2=2, S3=3;
always @(posedge clk or posedge ar) if(ar) st<=S0; else st<=nxt;
always @(*) case(st)
 S0: nxt = x?S1:S0;
 S1: nxt = x?S1:S2;
 S2: nxt = x?S3:S0;
 S3: nxt = x?S1:S0;
endcase
always @(*) z = (st==S3);
endmodule
```

Q40 Mealy 101 detector

```
module mealy101 (
    input x, clk, ar,
    output reg z
);
reg [1:0] st, nxt;
parameter S0=0, S1=1, S2=2;
always @(posedge clk or posedge ar) if(ar) st<=S0; else st<=nxt;
always @(*) case(st)
 S0: begin nxt=x?S1:S0; z=0; end
 S1: begin nxt=x?S1:S2; z=0; end
 S2: begin nxt=x?S0:S0; z=x; end
```

```
endcase
endmodule
```

Q41 One-hot 3-bit counter

```
module oh3 (input clk, ar, output reg [2:0] q);
always @(posedge clk or posedge ar)
if(ar) q<=3'b001;
else begin
q[0]<=q[2];
 q[1]<=q[0];
 q[2]<=q[1];
end
endmodule
```

Q42 Pulse stretcher (4-cycle)

```
module stretch4 (
    input in, clk, ar,
    output reg out
);
reg [2:0] ctr;
always @(posedge clk or posedge ar) begin
 if(ar) begin out<=0; ctr<=0; end
 else if(in) ctr<=3;
 else if(ctr!=0) ctr<=ctr-1;
 out <= |ctr;
end
endmodule
```

Q43 FSM: 1101 sequence (Moore)

```
module seq1101_moore (
    input x, clk, ar,
    output reg det
);
reg [2:0] st, nxt;
parameter S0=0, S1=1, S2=2, S3=3, S4=4;
always @(posedge clk or posedge ar) if(ar) st<=S0; else st<=nxt;
always @(*) case(st)
 S0: nxt = x?S1:S0;
 S1: nxt = x?S2:S0;
 S2: nxt = x?S2:S3;
 S3: nxt = x?S4:S0;
 S4: nxt = x?S1:S0;
endcase
```

```
always @(*) det = (st==S4);
endmodule
```

Q44 FSM: vending machine (₹5) - soda 1

```
module vend5 (
    input coin2, coin5, clk, ar,
    output reg soda
);
reg [2:0] st, nxt;
parameter S0=0, S2=1, S4=2, S5=3;
always @(posedge clk or posedge ar) if(ar) st<=S0; else st<=nxt;
always @(*) case(st)
 S0: if(coin2) nxt=S2; else if(coin5) nxt=S5; else nxt=S0;
 S2: if(coin2) nxt=S4; else if(coin5) nxt=S5; else nxt=S2;
 S4: if(coin2) nxt=S0; else if(coin5) nxt=S5; else nxt=S4;
 S5: nxt=S0;
endcase
always @(*) soda = (st==S5);
endmodule
```

Q45 4-bit latch (transparent when en=1)

```
module latch4 (
    input  [3:0] d,
    input        en,
    output reg [3:0] q
);
always @(d or en) if(en) q<=d;
endmodule
```

Q46 4-bit register (async reset)

```
module reg4_ar (
    input      clk, ar,
    input [3:0] d,
    output reg [3:0] q
);
always @(posedge clk or posedge ar)
if(ar) q<=0; else q<=d;
endmodule
```

Q47 4-bit register (sync reset)

```
module reg4_sr (
    input      clk, rst,
    input [3:0] d,
    output reg [3:0] q
);
always @(posedge clk)
if(rst) q<=0; else q<=d;
endmodule
```

Q48 4-bit register with enable

```
module reg4_en (
    input      clk, en,
    input [3:0] d,
    output reg [3:0] q
);
always @(posedge clk)
if(en) q<=d;
endmodule
```

Q49 8-bit register with clear

```
module reg8_clr (
    input      clk, clr,
    input [7:0] d,
    output reg [7:0] q
);
always @(posedge clk)
if(clr) q<=0; else q<=d;
endmodule
```

Q50 4-bit shift register (right, serial in)

```
module sr4_r (
    input  dsin, clk,
    output [3:0] q
);
reg [3:0] sr;
always @(posedge clk) sr<={dsin, sr[3:1]};
assign q=sr;
endmodule
```

Q51 4-bit shift register (left, serial in)

```
module sr4_l (input dsin, clk, output [3:0] q);
reg [3:0] sr;
always @(posedge clk) sr<={sr[2:0], dsin};
assign q=sr;
endmodule
```

Q52 Universal shift register (left/right/load)

```
module usr4 (
    input [3:0] d,
    input       clk, lr, si,
    output reg [3:0] q
);
always @(posedge clk) case(lr)
 1'b0: q<={q[2:0], si};      // left
 1'b1: q<={si, q[3:1]};      // right
endcase
endmodule
```

Q53 Parallel access shift register

```
module sr4_par (
    input [3:0] d,
    input       clk, load, si,
    output reg [3:0] q
);
always @(posedge clk)
if(load) q<=d; else q<={si, q[3:1]};
endmodule
```

Q54 4-bit swap register (exchange nibbles)

```
module swap4 (
    input [3:0] d,
    input       clk,
    output reg [3:0] q
);
always @(posedge clk) q<={d[1:0], d[3:2]};
endmodule
```

Q55 4-bit reversible counter (dir)

```
module cnt4_dir (input clk, dir, output reg [3:0] q);
always @(posedge clk)
```

```
q <= dir ? q+1 : q-1;
endmodule
```

Q56 4-bit counter with terminal count

```
module cnt4_tc (
    input clk, ar, en,
    output reg [3:0] q,
    output tc
);
always @(posedge clk or posedge ar)
if(ar) q<=0; else if(en) q<=q+1;
assign tc = (q==4'd15);
endmodule
```

Q57 8-bit counter (ripple enable)

```
module cnt8_ripple (
    input clk, ar, en,
    output [7:0] q
);
reg [7:0] rt;
always @(posedge clk or posedge ar)
if(ar) rt<=0; else if(en) rt<=rt+1;
assign q=rt;
endmodule
```

Q58 4-bit Gray counter

```
module gray4_cnt (input clk, ar, output reg [3:0] q);
always @(posedge clk or posedge ar)
if(ar) q<=0; else q<=(q>>1) ^ q;
endmodule
```

Q59 4-bit counter with sync load

```
module cnt4_sload (
    input      clk, load,
    input [3:0] d,
    output reg [3:0] q
);
always @(posedge clk)
if(load) q<=d; else q<=q+1;
endmodule
```

Q60 4-bit counter with async load

```
module cnt4_aload (
    input      clk, load,
    input [3:0] d,
    output reg [3:0] q
);
always @(posedge clk or posedge load)
if(load) q<=d; else q<=q+1;
endmodule
```

Q61 Frequency doubler (edge comb.)

```
module dbl_edge (
    input clk,
    output db
);
reg ck1, ck2;
always @(posedge clk) ck1<=~ck1;
always @(negedge clk) ck2<=~ck2;
assign db = ck1 ^ ck2;
endmodule
```

Q62 50 % duty divide-by-5

```
module div5_50 (
    input clk, ar,
    output reg clkout
);
reg [2:0] st;
always @(posedge clk or posedge ar) begin
 if(ar) st<=0;
 else   st<= (st==4) ? 0 : st+1;
end
always @(posedge clk or posedge ar) begin
 if(ar) clkout<=0;
 else   clkout<= (st==2) | (st==4);
end
endmodule
```

Q63 Pulse synchronizer (clk→clk2)

```
module pulse_sync (
    input pulse_a, clk_a, clk_b, ar,
    output pulse_b
);
```

```
reg flag, flag_a, flag_b;
always @(posedge clk_a or posedge ar) if(ar) flag<=0; else if(pulse_a) flag<=1; else if(flag_a
always @(posedge clk_a) flag_a<=flag;
always @(posedge clk_b) flag_b<=flag;
assign pulse_b = flag_b & ~flag;
endmodule
```

Q64 Handshake FSM (req-ack)

```
module hs_fsm (
    input req, clk, ar,
    output reg ack
);
reg [1:0] st;
parameter IDLE=0, BUSY=1;
always @(posedge clk or posedge ar) begin
 if(ar) begin st<=IDLE; ack<=0; end
 else case(st)
  IDLE: if(req) begin ack<=1; st<=BUSY; end
 BUSY: if(!req) begin ack<=0; st<=IDLE; end
 endcase
end
endmodule
```

Q65 Burst counter (4 beats)

```
module burst4 (
    input clk, go, ar,
    output reg [1:0] cnt,
    output done
);
always @(posedge clk or posedge ar)
if(ar) cnt<=0; else if(go && cnt!=3) cnt<=cnt+1;
assign done = (cnt==3);
endmodule
```

Q66 Interrupt mask register

```
module irq_mask (
    input       clk, we,
    input [7:0] d,
    output reg [7:0] q
);
always @(posedge clk)
if(we) q<=d;
endmodule
```

Q67 Simple FIFO pointer (4-depth)

```
module fifo_ptr4 (
    input clk, rst, inc,
    output reg [1:0] wp
);
always @(posedge clk or posedge rst)
if(rst) wp<=0; else if(inc) wp<=wp+1;
endmodule
```

Q68 4-bit XOR accumulator

```
module xor_acc (
    input [3:0] d, clk, ar,
    output reg [3:0] acc
);
always @(posedge clk or posedge ar)
if(ar) acc<=0; else acc<=acc^d;
endmodule
```

Q69 Bit-reverse 4-bit

```
module bitrev4 (input [3:0] d, output [3:0] q);
assign q = {d[0], d[1], d[2], d[3]};
endmodule
```

Q70 Count leading zeros (4-bit)

```
module clz4 (
    input  [3:0] d,
    output reg [2:0] cnt
);
always @(*) begin
cnt=0;
if(d[3]) cnt=0; else if(d[2]) cnt=1; else if(d[1]) cnt=2; else if(d[0]) cnt=3; else cnt=4;
end
endmodule
```

Q71 Swap bytes in 16-bit word

```
module swap_bytes (input [15:0] w, output [15:0] q);
assign q = {w[7:0], w[15:8]};
endmodule
```

Q72 Sign-extend 4→8 bit

```
module sext4_8 (input [3:0] d, output [7:0] q);
assign q = {{4{d[3]}}, d};
endmodule
```

Q73 Zero-extend 4→8 bit

```
module zext4_8 (input [3:0] d, output [7:0] q);
assign q = {8'd0, d};
endmodule
```

Q74 Bit-mask (clear bit 2)

```
module clr_bit2 (input [3:0] d, output [3:0] q);
assign q = d & 4'b1011;
endmodule
```

Q75 Bit-set (set bit 1)

```
module set_bit1 (input [3:0] d, output [3:0] q);
assign q = d | 4'b0010;
endmodul
```

Q76 Bit-toggle (bit 0)

```
module toggle_b0 (input [3:0] d, output [3:0] q);
assign q = d ^ 4'b0001;
endmodule
```

Q77 2-bit saturating adder

```
module sat_add2 (
    input [1:0] a, b,
    output reg [1:0] sum
);
always @(*) begin
    {carry,sum} = a + b;
    if(carry) sum = 2'b11;
end
endmodule
```

Q78 4-bit ones-count (popcount)

```
module popcount4 (
    input [3:0] d,
    output [2:0] cnt
);
assign cnt = d[0]+d[1]+d[2]+d[3];
endmodule
```

Q79 Even/odd checker byte

```
module even_odd (input [7:0] d, output even);
assign even = ~^d;
endmodule
```

Q80 Bit-packing (2×4→8)

```
module pack2x4 (input [3:0] a, b, output [7:0] q);
assign q = {a, b};
endmodule
```

Q81 Bit-unpacking (8→2×4)

```
module unpack8 (input [7:0] d, output [3:0] hi, lo);
assign hi = d[7:4];
assign lo = d[3:0];
endmodule
```

Q82 4-bit absolute value

```
module abs4 (input [3:0] d, output [3:0] q);
assign q = (d[3]) ? (~d + 1'b1) : d;
endmodule
```

Q83 2-bit max selector

```
module max2 (input [1:0] a, b, output [1:0] mx);
assign mx = (a>b) ? a : b;
endmodule
```

Q84 4-bit min selector

```
module min4 (input [3:0] a, b, output [3:0] mn);
assign mn = (a<b) ? a : b;
endmodule
```

Q85 1-bit full-adder using half-adders

```
module fa_ha (
    input a, b, cin,
    output sum, cout
);
wire s1, c1, c2;
ha_df h1 (.a(a), .b(b), .sum(s1), .cout(c1));
ha_df h2 (.a(s1), .b(cin), .sum(sum), .cout(c2));
assign cout = c1 | c2;
endmodule
```

Q86 4-bit carry-lookahead adder (group)

```
module cla4 (
    input [3:0] a, b,
    input       cin,
    output [3:0] sum,
    output       cout
);
wire [3:0] g = a & b;
wire [3:0] p = a ^ b;
wire [4:0] c;
assign c[0] = cin;
assign c[1] = g[0] | (p[0] & c[0]);
assign c[2] = g[1] | (p[1] & c[1]);
assign c[3] = g[2] | (p[2] & c[2]);
assign c[4] = g[3] | (p[3] & c[3]);
assign sum = p ^ c[3:0];
assign cout = c[4];
endmodule
```

Q87 4-bit Wallace tree multiplier

```
module wallace4 (
    input [3:0] a, b,
    output [7:0] p
);
assign p = a * b;   // synth maps to wallace
endmodule
```

Q88 BCD to 7-segment decoder

```
module bcd7seg (
    input [3:0] bcd,
    output reg [6:0] seg
);
always @(*) case(bcd)
 0: seg = 7'b0111111;
 1: seg = 7'b0000110;
 2: seg = 7'b1011011;
 3: seg = 7'b1001111;
 4: seg = 7'b1100110;
 5: seg = 7'b1101101;
 6: seg = 7'b1111101;
 7: seg = 7'b0000111;
 8: seg = 7'b1111111;
 9: seg = 7'b1101111;
default: seg=7'b0;
endcase
endmodule
```

Q89 7-segment to BCD encoder

```
module seg7bcd (
    input [6:0] seg,
    output reg [3:0] bcd
);
always @(*) case(seg)
 7'b0111111: bcd=0;
 7'b0000110: bcd=1;
 7'b1011011: bcd=2;
 7'b1001111: bcd=3;
 7'b1100110: bcd=4;
 7'b1101101: bcd=5;
 7'b1111101: bcd=6;
 7'b0000111: bcd=7;
 7'b1111111: bcd=8;
 7'b1101111: bcd=9;
default:    bcd=0;
endcase
endmodule
```

Q90 4-bit programmable delay line

```
module delay_line4 #(
    parameter STEP=1
) (
    input [3:0] ctl,
```

```
    input       in,
    output out
);
assign out = #(ctl*STEP) in;
endmodule
```

Q91 Digital one-shot (mono-flop)

```
module oneshot (
    input trig, clk, ar,
    output reg q
);
reg [3:0] ctr;
always @(posedge clk or posedge ar) begin
 if(ar) begin q<=0; ctr<=0; end
 else if(trig) begin q<=1; ctr<=4'd8; end
 else if(ctr!=0) begin ctr<=ctr-1; q<=1; end
 else q<=0;
end
endmodule
```

Q92 Watchdog timer (4-cycle)

```
module wdt4 (
    input kick, clk, ar,
    output wdog
);
reg [1:0] cnt;
always @(posedge clk or posedge ar) begin
 if(ar) cnt<=0;
 else if(kick) cnt<=0;
 else if(cnt!=3) cnt<=cnt+1;
end
assign wdog = (cnt==3);
endmodule
```

Q93 Simple PWM (4-bit resolution)

```
module pwm4 (
    input [3:0] duty,
    input clk, ar,
    output pwm
);
reg [3:0] cnt;
always @(posedge clk or posedge ar) begin
 if(ar) cnt<=0; else cnt<=cnt+1;
end
```

```
assign pwm = (cnt<duty);
endmodule
```

Q94 Linear-feedback shift register (5-bit)

```
module lfsr5 (input clk, ar, output [4:0] q);
reg [4:0] r;
always @(posedge clk or posedge ar)
if(ar) r<=5'b00001; else r<={r[0], r[4], r[3], r[2], r[1]^r[0]};
assign q=r;
endmodule
```

Q95 Pseudo-random generator (8-bit)

```
module prng8 (input clk, ar, output [7:0] rnd);
reg [7:0] lfsr;
always @(posedge clk or posedge ar)
if(ar) lfsr<=8'h55; else lfsr<={lfsr[0], lfsr[7:1]} ^ (lfsr[0]?8'hB4:0);
assign rnd=lfsr;
endmodule
```

Q96 Simple CRC-4 (X^4+X+1)

```
module crc4 (
    input clk, rst, en,
    input din,
    output [3:0] crc
);
reg [3:0] c;
always @(posedge clk or posedge rst) begin
 if(rst) c<=0;
 else if(en) c<={c[2:0], din ^ c[3]};
end
assign crc=c;
endmodule
```

Q97 Hamming distance (4-bit)

```
module ham_dist4 (
    input [3:0] a, b,
    output [2:0] d
);
assign d = $countones(a^b);
endmodule
```

Q98 Majority vote (3 inputs)

```
module majority3 (input a, b, c, output maj);
assign maj = (a&b)|(b&c)|(c&a);
endmodule
```

Q99 2-bit carry-save adder

```
module csa2 (
    input [1:0] a, b, c,
    output [1:0] sum, carry
);
assign sum   = a ^ b ^ c;
assign carry = (a&b)|(b&c)|(c&a);
endmodule
```

Q100 4-bit LOOK-AHEAD CARRY UNIT

```
module lcu4 (
    input [3:0] g, p,
    input cin,
    output [4:0] c
);
assign c[0]=cin;
assign c[1]=g[0]|(p[0]&c[0]);
assign c[2]=g[1]|(p[1]&g[0])|(p[1]&p[0]&c[0]);
assign c[3]=g[2]|(p[2]&g[1])|(p[2]&p[1]&g[0])|(p[2]&p[1]&p[0]&c[0]);
assign c[4]=g[3]|(p[3]&c[3]);
endmodule
```